

P2905R0

Runtime format strings

Published Proposal, 2023-05-28

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Problems

3 Proposal

4 Impact on existing code

5 Wording

6 Implementation

References

Informative References

"Temporary solutions often become permanent problems." — Craig Bruce

§ 1. Introduction

[P2216] "std::format improvements" introduced compile-time format string checks which, quoting Barry Revzin, "is a fantastic feature" ([P2757]). However, due to resource constraints it didn't provide a good API for using formatting functions with format strings not known at compile time. As a workaround one could use type-erased API which has never been designed for that. This severely undermined safety and led to poor user experience. This paper fixes the issue by proposing direct support for runtime format strings which has been long available in the {fmt} library.

§ 2. Problems

[P2216] "std::format improvements" introduced compile-time format string checks for `std::format`. This obviously requires format strings be known at compile time. However, there are some use cases where format strings are only known at runtime, e.g. when translated through gettext ([GETTEXT]). One possible workaround is using type-erased formatting functions such as `std::vformat`:

```
std::string str = translate("The answer is {}.");
std::string msg = std::vformat(str, std::make_format_args(42));
```

This is not a great user experience because the type-erased API was designed to avoid template bloat and should only be used by formatting function writers and not by end users.

Such misuse of the API also introduces major safety issues illustrated in the following example:

```
std::string str = "{}";
std::filesystem::path path = "path/etic/experience";
auto args = std::make_format_args(path.string());
std::string msg = std::vformat(str, args);
```

This innocent-looking code exhibits undefined behavior because format arguments store a reference to a temporary which is destroyed before use. This has been discovered and fixed in [FMT] which now rejects such code at compile time.

§ 3. Proposal

This paper proposes adding the `std::runtime_format` function to explicitly mark a format string as a runtime one and opt out of compile-time format string checks.

Before	After
<code>std::vformat(str, std::make_format_args(42));</code>	<code>std::format(std::runtime_format(str), 42);</code>

This improves usability and makes the intent more explicit. It can also enable detection of some lifetime errors for arguments ([P2418]). This API has been available in {fmt} since `constexpr`-based format string checks were introduced ~2 years ago and usage experience was very positive. In a large codebase with > 100k calls of `fmt::format` only ~0.1% use `make_format_args`.

This paper also proposes changing `make_format_args` to take lvalue references instead of rvalue references, rejecting problematic code:

```
std::filesystem::path path = "path/etic/experience";
auto args = std::make_format_args(path.string()); // ill-formed
```

This has also been implemented in {fmt} catching some bugs even though the pattern of using `make_format_args` has never been suggested as a way to pass runtime format strings there. If left unchanged this will be a major safety hole in the standard formatting facility.

In the standard itself `make_format_args` is already called with lvalue references only, e.g. [\[format.functions\]](#):

```
template<class... Args>
    string format(format_string<Args...> fmt, Args&&... args);
```

Effects: Equivalent to:

```
return vformat(fmt.str, make_format_args(args...));
```

Notice that there is intentionally no forwarding of `args` so the switch from forwarding to lvalue references is effectively a noop there.

§ 4. Impact on existing code

Rejecting temporaries in `make_format_args` is an (intentionally) breaking change.

Searching GitHub for calls of `std::make_format_args` using the following query

```
"std::make_format_args" language:c++ -path:libstdc -path:libcxx -path:include/c++
```

returned only 844 results at the time of writing. For comparison, similar search returned 165k results for `fmt::format` and 7.3k for `std::format`. Such low usage is not very surprising because `std::format` is not widely available yet.

At least 452 of these call sites use `make_format_args` as intended and will require no changes:

```
std::vformat_to(std::back_inserter(c), fmt.get(), std::make_format_args(args...));
```

72 of remaining calls can be trivially fixed by removing unnecessary forwarding.

This leaves only 320 cases most of which will continue to work and the ones that pass temporaries can be easily fixed by either switching to `std::runtime_format` or by storing a temporary in a variable.

§ 5. Wording

Change in [\[format.syn\]](#):

```
namespace std {
    ...

    // [format.fmt.string], class template basic_format_string
    template<class charT, class... Args>
        struct basic_format_string;

    template<class... Args>
        using format_string = basic_format_string<char, type_identity_t<Args>...>;
    template<class... Args>
```

```

        using wformat_string = basic_format_string<wchar_t, type_identity_t<Args>...>;

        template<class charT> struct runtime-format-string { // exposition-only
            basic_string_view<charT> str;
        };

        runtime-format-string<char> runtime_format(string_view fmt) { return {fmt}; }
        runtime-format-string<wchar_t> runtime_format(wstring_view fmt) { return {fmt}; }

        ...

        template<class Context = format_context, class... Args>
        format_arg_store<Context, Args...>
        make_format_args(Args&... fmt_args);
        template<class... Args>
        format_arg_store<wformat_context, Args...>
        make_wformat_args(Args&... args);

        ...
    }

```

Change in `[format.fmt.string]`:

```

namespace std {
    template<class charT, class... Args>
    struct basic_format_string {
    private:
        basic_string_view<charT> str;           // exposition only

    public:
        template<class T> constexpr basic_format_string(const T& s);
        basic_format_string(runtime-format-string<charT> s) : str(s.str) {}

        constexpr basic_string_view<charT> get() const noexcept { return str; }
    };
}

```

Change in `[format.arg.store]`:

```

template<class Context = format_context, class... Args>
format_arg_store<Context, Args...> make_format_args(Args&... fmt_args);

```

2 *Preconditions*: The type `typename Context::template formatter_type<remove_cvrefconst_t<T>i>` meets the *BasicFormatter* requirements (`[formatter.requirements]`) for each `Ti` in `Args`.

...

```

template<class... Args>

```

```
format_arg_store<wformat_context, Args...> make_wformat_args(Args&&... args);
```

§ 6. Implementation

The proposed API has been implemented in the `{fmt}` library ([FMT]).

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. [The fmt library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[GETTEXT]

Free Software Foundation. [gettext](https://www.gnu.org/software/gettext/). URL: <https://www.gnu.org/software/gettext/>

[P2216]

Victor Zverovich. [std::format improvements](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2216r3.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2216r3.html>

[P2418]

Victor Zverovich. [Add support for `std::generator`-like types to `std::format`](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2418r2.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2418r2.html>

[P2757]

Barry Revzin. [Type-checking format args](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2757r1.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2757r1.html>